Longest Common Subsequence

Presented by

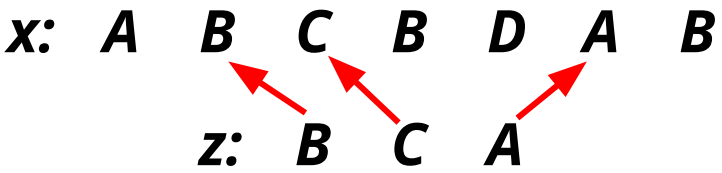
Rashed Hassan Siam

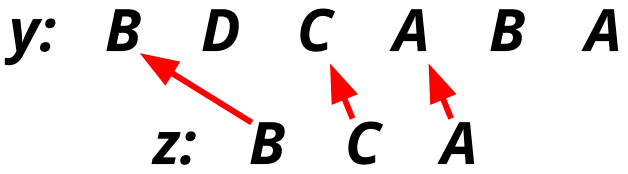
Outline

- ***Subsequence and Common Subsequence***
- ***The Longest Common Subsequence (LCS) Problem***
- ***Brute Force (Naive) Method and Analysis***
- ***Dynamic Programming Method and Analysis***
- ***Applications***

Subsequence and Common Subsequence

- A subsequence of a given sequence is just the given sequence with zero or more elements left out.
- Given two sequences ***x*** and ***y***, we say that a sequence ***z*** is a common subsequence of ***x*** and ***y*** if ***z*** is a subsequence of both ***x*** and ***y***.





So, z is a common subsequence of both x and y.

The Longest Common Subsequence (LCS) Problem

- In the LCS problem, we are given two sequences ***x[1 … m]*** and ***y[1 … n]*** and we find a maximum length common subsequence of ***x*** and ***y***.
- Prominent ways to achieve this are:
  - ***Brute Force (Naive) Method*** (Less efficient)
  - ***Dynamic Programming*** (More efficient)

Using Brute Force (Naive) Method

- We check every subsequence of ***x[1 … m]*** to see if it is also a subsequence of ***y[1 … n]***.
- ***Analysis:***
  - There are 2^m subsequences of ***x***.
  - Checking on ***y*** takes ***O(n)*** time per subsequence.
  - Worst-case running time = $O(n * 2^m)$ [Exponential time]

Towards A Better Solution: Dynamic Programming

- Let ***c[i, j]*** be the length of an LCS of the sequences x_i and y_j (upto their ***i-th*** and ***j-th*** indices respectively).
- Now the LCS problem gives us the recursive formula:

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(c[i, j - 1], c[i - 1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

Dynamic Programming: Determining LCS Length

LCS-Length (x, y)

1. *m = x.length*
2. *n = y.length*
3. *let b[1 … m, 1 … n] and c[0 … m, 0 … n] be new tables*
4. *for i = 1 to m*
5. *c[i, 0] = 0*
6. *for j = 0 to n*
7. *c[0, j] = 0*

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0						
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Dynamic Programming: Determining LCS Length (Contd.)

```
8.    for i = 1 to m
9.        for j = 1 to n
10.           if x_i == y_j
11.               c[i, j] = c[i - 1, j - 1] + 1
12.               b[i, j] = " ↖ "
13.           else if c[i - 1, j] ≥ c[i, j - 1]
14.               c[i, j] = c[i - 1, j]
15.               b[i, j] = " ↑ "
16.           else  c[i, j] = c[i, j - 1]
17.               b[i, j] = " ← "
18.   return c[m][n] and b
```

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑0	↑0	↑0	↖1	←1	↖1
2	B	0	↖1	←1	←1	↑1	↖2	←2
3	C	0	↑1	↑1	↖2	←2	↑2	↑2
4	B	0	↖1	↑1	↑2	↑2	↖3	←3
5	D	0	↑1	↖2	↑2	↑2	↑3	↑3
6	A	0	↑1	↑2	↑2	↖3	↑3	↖4
7	B	0	↖1	↑2	↑2	↑3	↖4	↑4

Dynamic Programming: Constructing An LCS

PRINT-LCS (b, x, i=x.length, j=y.length)

1. *if i == 0 or j == 0*
2. *return*
3. *if b[i, j] == " ↖ "*
4. *PRINT-LCS (b, x, i - 1, j - 1)*
5. *print x_i*
6. *else if b[i, j] == " ↑ "*
7. *PRINT-LCS (b, x, i - 1, j)*
8. *else PRINT-LCS (b, x, i, j - 1)*

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑0	↑0	↑0	↖1	←1	↖1
2	B	0	↖1	←1	←1	↑1	↖2	←2
3	C	0	↑1	↑1	↖2	←2	↑2	↑2
4	B	0	↖1	↑1	↑2	↑2	↖3	←3
5	D	0	↑1	↖2	↑2	↑2	↑3	↑3
6	A	0	↑1	↑2	↑2	↖3	↑3	↖4
7	B	0	↖1	↑2	↑2	↑3	↖4	↑4

Dynamic Programming: Analysis

- The running time of the ***LCS-Length*** procedure is ***O(mn)***, since each table entry takes ***O(1)*** time to compute.
- The procedure ***PRINT-LCS*** takes time ***O(m + n)***, since it decrements at least one of ***i*** and ***j*** in each recursive call.
- So, the overall running time to count the length and construct an LCS is: ***O(mn) + O(m + n)***
- It is substantially better than the exponential running time of the brute force method.

Applications

- Finding similarities among DNA strands based on their Nucleotide base subsequences.
  - For example, suppose a DNA strand S_3 = ***GTCGTCGGAAGCCGGCCGAA*** is a subsequence of the strands S_1 = ***ACCGGTCGAGTGCGCGGAAGCCGGCCGAA*** and S_2 = ***GTCGTTCGGAATGCCGTTGCTCTGTAAA***. The longer the strand S_3 we can find, the more similar S_1 and S_2 will be.
- Similar applications of LCS include, text similarity, paraphrase detection, code similarity detection, etc.

Summary

- In the ***LCS (Longest Common Subsequence)*** problem, we try to find a maximum length common subsequence of two given sequences.
- The brute force method achieves this in exponential time of $O(n * 2^m)$, which is inefficient.
- As a better solution, the dynamic programming approach gives a running time of ***O(mn) + O(m + n)***, which is much better in terms of efficiency.
- Used in checking DNA strand similarity, text similarity, code similarity, etc.

Reference

- ***Introduction to Algorithms, Third Edition***
  - ***Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein***

Thank You!

Any Questions?